STARK Fundamentals — Review Notes

Merged: conceptual notes + end-to-end pipeline walkthrough. Organized for spaced review — Parts I–IV are the conceptual substrate, Part V is the synthesis that uses all of it, Parts VI–VII are the protocol layer wrapped around it.

Part I — Why STARKs need only a hash function

- A STARK is built in **two layers**; nearly all its security lives in the layer with no cryptography in it.
- **Layer 1 (polynomial IOP) is information-theoretic.** Soundness rests on facts about polynomials over finite fields, not on hardness assumptions. An unbounded prover still can't cheat it — the obstacle isn't difficulty, it's that the lie would have to be true at too many points at once.
- In this layer the prover hands over **oracles**: queryable objects assumed fixed before any query. That assumption is the only thing doing work.
- **Layer 2 (compilation) is where cryptography enters**, and only to make those oracles real:
  - **Merkle trees** → vector commitment. Binding comes from collision resistance; answering a query inconsistently with the committed root means producing a hash collision.
  - **Fiat–Shamir** → removes interaction. Needs the hash to behave like a random oracle.
- Both jobs are served by one primitive → the only cryptographic ingredient is a collision-resistant hash. Provably post-quantum under an idealized hash model.
- **Why SNARKs need more:** KZG commits by evaluating at a secret point hidden in the exponent → needs hard discrete log, a pairing, and a trusted setup whose randomness must be destroyed. Its degree bound comes free from the finite SRS; a Merkle tree has no algebraic structure, **which is exactly why STARKs need FRI**. FRI is the price of a commitment scheme with no math in it.
- Side benefit: the arithmetization field is independent of any hard problem → choose small fast fields (Goldilocks, Mersenne-31). Pairing systems are stuck at ~255 bits because security lives there.
- **Caveats:** the random oracle model is a strong idealization, not a falsifiable reduction. FRI's concrete soundness at aggressive rates rests on unproven proximity-gap conjectures. Zero-knowledge does **not** come from the commitment hiding anything — it comes from randomizing the trace.

Part II — Finite fields

- $\mathbb{F}_p$ = integers mod prime p. Ordinary algebra remains fully legal; that's the entire mental shift.
- **Primality is what buys division.** Mod 6, $2 \times 3 = 0$ → zero divisors → $1/2$ can't exist.

Prime modulus → no zero divisors → every nonzero element invertible.

- $\mathbb{F}_p^\times$ is **cyclic of order** $p-1$: one generator sweeps all nonzero elements; for each $n \mid p-1$ there's exactly one subgroup of order n, generated by an n**-th root of unity** ω ($\omega^n = 1$).
- **Why roots of unity matter:** put the trace on $D = \{1, \omega, \omega^2, \ldots, \omega^{n-1}\}$ and "next step" becomes **multiplication by ω** — one field operation. That's what lets a transition rule be *one* polynomial identity instead of n separate checks.
- **STARK-friendly primes** have $p-1$ divisible by a large power of 2, so 2^k-order roots of unity exist → radix-2 FFT possible.
- **Extension fields** $\mathbb{F}_{p^k} = \mathbb{F}_p[X]/(\text{irreducible})$: needed when the base field is small, since a challenge drawn from it has too little entropy. Trace stays in the cheap base field, challenges come from the extension.

Field capacity as a budget

- Everything downstream — trace domain, evaluation domain, challenge space — is drawn from the same universe of $p-1$ nonzero elements. The prime is not just an arithmetic choice; it's the **capacity constraint** on domain sizes and the **entropy constraint** on challenges.
- The two constraints pull apart, which is why they get solved separately: domain size is bounded by the **2-adicity** of $p-1$, while challenge entropy is bounded by $|\mathbb{F}|$ and is repaired by sampling from an extension field rather than by enlarging p.

Part III — The two facts

Fact 1 — a degree-d polynomial has at most d roots

- Each root gives a linear factor; degree d can't split into more than d of them. **"At most," not "exactly"** — X^2+1 over $\mathbb{F}_7$ has zero roots.
- The operative idea is the **ceiling**: low degree = few places a zero can hide.
- **Consequence that powers everything:** two distinct degree-$< d$ polynomials agree only where their difference vanishes, and that difference is itself low degree → they agree in **fewer than d places**.
- So checking one random point ≈ checking all of them. In $\mathbb{F}_7$: $f = X^2+3X+2$ vs. $h = X^2+5X+3$ differ by $2X+1$, which vanishes only at $X=3$ → a liar escapes 1 time in 7.
- No hardness assumption anywhere in that argument. **This is literally what the information-theoretic layer is made of.**

Fact 2 — $f(a) = y \iff (X-a) \mid (f(X)-y)$

- Division with remainder: $f(X) = q(X)(X-a) + r$, and since $\deg(X-a) = 1$, r **is a constant**.
- Plug in $X = a$: the $q(a)$ term dies → $r = f(a)$. Evaluating and dividing are the same operation.

- **The key reframing: a lie doesn't produce a *wrong* quotient, it produces a *high-degree* one.** Honest division → a genuine polynomial of low degree. Dishonest → a rational function with a pole, whose values on the remaining points interpolate to near-maximal degree.
- So the verifier never checks the original statement — only a **degree bound**. Checking degree bounds is exactly what FRI does.
- **This is the converter** that turns the whole tutorial's arc ("computational claims → low-degree claims") into a mechanism.
- Boundary constraint in practice: "trace starts at 1" → prover exhibits $\frac{t(X)-1}{X-1}$ and proves it's low degree.
- **Generalization — vanishing polynomial:** $Z_S(X) = \prod_{s \in S}(X - s)$. "$f$ vanishes on all of S" = "$Z_S \mid f$". Transition constraints use this: one division enforces every step.
- When S = roots of unity, $Z_S(X) = X^n - 1$ — sparse, evaluable in $\log n$ time instead of n. **Another reason that domain is chosen.**

Part IV — Why low-degree claims matter (three strands)

1. **Degree is the scarcity that makes a claim non-vacuous.** Without a degree bound, "vanishes on the trace domain" says nothing — interpolate anything through n points.
2. **Fact 2 already converted every claim into a degree claim.** Boundary conditions, transition rules, the whole execution — after arithmetization only *one kind of claim* remains. That collapse is why FRI is the only machinery needed on top of the hash.
3. **Low degree makes random sampling work, quantitatively.** With rate $\rho = d/n = 1/4$, a liar's table differs from every valid codeword in $\geq 3/4$ of positions → one query catches with prob. $3/4$; 20 queries → $\approx 1 - 10^{-12}$.

- **Rate is the main tuning knob:** lower rate → better soundness per query, but larger domain → more prover work and bigger Merkle trees.

> ⚠ **Precision check on strand 3.** The clean $1 - \rho$ bound holds against a table that *is itself a different codeword*. A table that is merely **close to** a valid codeword without being one is the hard case, and bounding a cheating prover there is exactly what the **proximity-gap conjectures** cover — the unproven assumption flagged in Part I. Keep the clean bound as intuition, but know it's the idealized version.

Part V — The end-to-end pipeline

This part is the synthesis: Parts II–IV supply every ingredient, and nothing new is assumed.

1. Foundation & field setup

- All arithmetic — trace steps, evaluations, constraints — happens in a finite prime field $\mathbb{F}_p$.
- p fixes the universe: $p - 1$ nonzero elements from which variables, domain points, and challenges are drawn. (→ Part II, *Field capacity as a budget*.)

2. Execution trace & arithmetization

- **Execution trace (matrix).** Run the computation normally and record it: a 2D table of T rows (steps), with columns holding the state registers over time. The witness is a free byproduct of doing the work — nothing is searched for. (→ Part VI, *Simplification 1.*)
- **Trace polynomials $t(X)$.** Interpolate each column over the trace domain of size T → polynomials of degree $< T$. The domain is the roots-of-unity subgroup, so "next row" is multiplication by ω.
- **Transition constraints $C(X)$.** Algebraic formulas asserting every step obeyed the program logic, written over $t_i(X)$ and $t_i(\omega X)$.
- **Constraint multiplier & max degree d.** A degree-c constraint applied to degree-$(< T)$ trace polynomials yields degree $\approx c \cdot T$. That product is the target degree bound d, and it is what sets the blowup requirement downstream.

> ⚠ **Missing step worth adding.** Multiple constraints are not proven separately. The verifier sends random coefficients and the prover forms a **random linear combination** of all constraints into one *composition polynomial*. Soundness of the batching is again Fact 1: if any constituent is nonzero, the combination is nonzero except with probability $\approx 1/|\mathbb{F}|$. This is also the second place extension-field challenges are needed.

3. Boundary enforcement & polynomial division

- **Vanishing polynomial $Z(X)$.** Zero at every point of the trace domain; equals $X^T - 1$ for the roots-of-unity domain. (→ Part III, Fact 2 generalization.)
- **Quotient polynomial $Q(X)$.** Divide the composition polynomial by $Z(X)$.
- **Algebraic scarcity.** An honest execution yields a genuine low-degree $Q(X)$ of degree $\approx d - T$. A dishonest step yields a rational function with poles, whose values interpolate to a high-degree mess approaching degree $N - 1$. **The lie is not a wrong value — it is a degree explosion.** (→ Part III, Fact 2, key reframing.)

> ⚠ **Operational gap.** This division cannot be performed in evaluation form on the trace domain, because Z is zero at every point there — every division is $0/0$. The quotient is computed on a **coset** gD, where $Z(g\omega^i) = g^T - 1$ is a nonzero *constant*, so the whole division costs one inversion plus N multiplications. Coset evaluation is not an optimization here; it's what makes the step defined at all.

4. Low-degree extension & domain blowup

- **Evaluation domain N.** Expand the domain by a blowup factor K (typically 4 or 8).
- **Mathematical bound.** $d < N$, and N must be available in the field.
- **Reed–Solomon codeword generation.** Evaluating the degree-d quotient over the size-N domain turns the evaluation table into a Reed–Solomon codeword. The table *is* the codeword; "polynomial" and "codeword" are two views of one object.
- **Soundness mechanism.**
  - Unconstrained high-degree polynomials fit any data — you can interpolate

anything through N points.

- Restricting to $d \ll N$ establishes a code rate $\rho = d/N$.
- Two distinct low-degree polynomials agree on fewer than d of the N points. (→ Part III, Fact 1.)
- So a liar's table differs from any valid codeword in at least $1 - \rho$ of positions (75% at $K = 4$), and spot-checking catches lies with high probability. (→ Part IV, strand 3, including its caveat.)

> ⚠ **Two corrections to the sizing.**
>
> 1. N is normally set as $K\times$ (trace length), not $K \times d$. The blowup is defined against the domain the trace lives on; d then has to fit under N with margin, which is what constrains how high the constraint degree c may go.
> 2. "$N \leq p - 1$" is too weak. N must be a power of two **dividing** $p - 1$ — i.e. bounded by the **2-adicity** of the field, not by its size. Goldilocks has 2-adicity 32 and BabyBear 27, so those are the real ceilings. Mersenne-31 has 2-adicity 1 and admits no standard NTT domain at all, which is the entire motivation for Circle STARKs.

5. Commitment & verification protocol

- **Merkle commitment.** Evaluate across all N points, hash the evaluations into a Merkle tree, send the root. Binding comes from collision resistance. (→ Part I, Layer 2.)
- **Low-degree proof (FRI).** Prove the committed N evaluations lie on (or near) a true low-degree polynomial. FRI's folding step $f_e + \alpha f_o$ is the FFT butterfly run in reverse.
- **Random sampling queries.** The verifier requests Merkle opening paths at a small number of random positions, checking consistency without re-running the computation. Query count × per-query soundness is what buys the 2^{-100} margin.
- **Then Fiat–Shamir** collapses all of the above into a single non-interactive message. (→ Part VII.)

Part VI — Interactive proof systems

- A proof here is a **conversation**, not a document. Payoff: the verifier checks without reading the whole thing — a liar can't be ready for every question.
- **"Convinced iff true" = two requirements.** Completeness: true → accept. Soundness: false → reject. Either is trivial alone (accept-everything, reject-everything); the design problem is both at once, cheaply.
- Soundness is **statistical** — real systems push cheating probability to ~2^{-100}.

Simplification 1: computational integrity

- Claims restricted to "program on input x gave output y."
- **The restriction is about the prover, not the verifier.** For an open conjecture the prover must first *discover* a proof. For computational integrity, **the witness is a free byproduct of doing the work** — you ran it, so you have the trace. Nothing to search for.

- **Two contrasts to keep straight:**
  - vs. **arbitrary mathematical claims** (the tutorial's literal contrast).
  - vs. **proving knowledge of a secret** (discrete log, preimage, graph coloring). CI is a claim about a *process*, not a secret.
- → ZK is an **optional add-on**. A rollup proving 10,000 public transactions hides nothing; it just compresses verification. "Validity proof" is often the more accurate term.

Simplification 2: public coins

- **Private-coin:** verifier's randomness is hidden and security *depends* on the hiding. E.g. graph non-isomorphism — a prover who sees the coin just reads off the answer and the protocol conveys nothing.
- **Public-coin:** messages are raw coin flips, nothing mixed in, safe to shout.
- **The distinction in one line:** private-coin needs randomness **hidden**; public-coin needs it **unpredictable at commitment time**. STARK challenges ("evaluate at 3", "fold with α") have no secret to protect — security comes from the **commitment coming first**.
- "Without loss of generality" = Goldwasser–Sipser: private coins buy no meaningful extra power.
- **Only public-coin protocols can be Fiat–Shamir'd** — you can't hash your way to a secret the prover mustn't know. Hence STARKs, Bulletproofs, Plonk are all designed public-coin from the start: interactivity is meant to be deleted later.

Part VII — Fiat–Shamir

- Replace each challenge with a **hash of the entire transcript so far** → proof becomes a single message.
- **Not a trust or open-source question.** The verifier **recomputes every challenge itself** from the roots in the proof and checks the openings against them. A fabricated challenge simply fails the recomputation.
- **Hashing over everything prior enforces the ordering:** revising a commitment changes the hash input → changes the challenge → invalidates whatever the prover was setting up. The dependency chain replaces the live verifier's timing.
- **Grinding** (the one residual attack): tweak a nonce, rehash, resample the challenge, repeat. Free retries from a distribution the prover can't steer.
- **"Security margin high enough"** = set query count and rate so per-challenge cheating probability is below ~2^{-100}, leaving no realistic grinding budget sufficient. (At 2^{-80} with 2^{40} attempts → 2^{-40} — remote, but the margin is visibly eroding.)
- **Proof-of-work grinding** as explicit countermeasure: require $\mathrm{hash}(\mathrm{transcript} \| \mathrm{nonce})$ to have ~20 leading zero bits. Honest prover pays 2^{20} once; a cheater pays it **per sampled challenge**, dividing their budget by a million.
- **The real assumption is the random oracle model** — this is the precise source of the "idealized model of the hash function" caveat on the post-quantum claim.

- **Most common implementation bug in the field:** "weak Fiat–Shamir" — forgetting to absorb a value, or hashing in the wrong order, lets a prover choose data *after* seeing the challenge meant to bind it. The math is airtight; transcript bookkeeping is where people slip.

Part VIII — Worked example: committing to a polynomial with a Reed–Solomon Merkle tree + FRI

Every number here was computed, not invented: field $\mathbb{F}_{17}$, SHA-256 truncated to 8 hex characters so the hashes fit on a line. It continues the squaring example: the honest quotient there was $Q(X) = 4X^3 + 15X^2 + 14X + 3$.

The goal. The prover wants to commit to Q and convince the verifier that "the thing I committed to is a polynomial of degree < 4" without sending Q. Three pieces do it: **Reed–Solomon encoding** (evaluate on a bigger domain), a **Merkle tree** (bind to those evaluations), and **FRI** (prove the evaluations are low degree by folding).

Step 1 — Reed–Solomon encode: evaluate on a larger domain

- Degree bound $d = 4$. Blowup $K = 2$ → evaluation domain of size $N = 8$, rate $\rho = 4/8 = 1/2$. (Real systems use $K \geq 4$; 2 keeps the tables short.)
- $\omega = 9$ has order 8 in $\mathbb{F}_{17}$, so the domain is

index i	0	1	2	3	4	5	6	7
$x = 9^i$	1	9	13	15	16	8	4	2
$Q(x)$	2	10	16	3	0	12	11	4

- This row of 8 values **is the Reed–Solomon codeword**. Only 4 of the values are free; the other 4 are forced by the degree bound. That redundancy is what makes lies detectable.
- **Note the pairing:** $9^4 = 16 = -1$, so index i and index $i + 4$ hold x and $-x$ (e.g. 13 and 4). FRI depends on this.
- (In a real STARK the domain is a coset $g \cdot \langle \omega \rangle$, because the quotient must be computed off the trace domain. FRI runs identically on a coset, since x and $-x$ still pair up.)

Step 2 — Merkle-commit to the codeword

Hash each value into a leaf, then hash pairs upward:

```
level 0 (leaves):  e9af7b7f 67470796 fbc05c36 aa4ca58e 8f714d24 69ba2dc8
ac5cc656 3e44a2d3
                   Q(1)    Q(9)    Q(13)   Q(15)   Q(16)   Q(8)    Q(4)
Q(2)
level 1:              cc36b119         cb46c7c6          e5cca044
7c0a2b78
level 2:                       dcb20c92                          dc7a368a
root:                                  4a4aa549
```

- The prover sends **only the root** `4a4aa549`. This is the commitment: 8 bytes standing for 8 field elements, binding by collision resistance.
- **What the root does *not* say:** anything about degree. A Merkle tree commits to *any* vector; it would hash 8 random numbers just as happily. The degree claim is FRI's job. (Part I: "FRI is the price of a commitment scheme with no math in it.")

Step 3 — FRI fold #1: halve the degree

- Split Q into even and odd parts: $Q(X) = Q_e(X^2) + X\,Q_o(X^2)$, where Q_e, Q_o have degree < 2.
- Both parts can be recovered **pointwise from a pair** $\{x, -x\}$ without knowing coefficients:

$$Q_e(x^2) = \frac{Q(x) + Q(-x)}{2}, \qquad Q_o(x^2) = \frac{Q(x) - Q(-x)}{2x}$$

- Fiat-Shamir challenge: $\alpha_1 = \text{hash}(\text{root}_0) \bmod 17 = 13$.
- The folded function is $Q_1(Y) = Q_e(Y) + \alpha_1 Q_o(Y)$: **degree < 2, on the squared domain** $\{1, 13, 16, 4\}$ (size 4). Rate is unchanged at $2/4 = 1/2$.

$y = x^2$	**1**	**13**	**16**	**4**
$Q_1(y)$	14	2	16	11

- Commit again: Merkle root `e35f743e`.

Check one entry by hand (index 2, from the pair $x = 13, -x = 4$, values 16 and 11):

- $Q_e = (16 + 11)/2 = 10 \cdot 9 = 5$ (since $2^{-1} = 9$)
- $Q_o = (16 - 11)/(2 \cdot 13) = 5/9 = 5 \cdot 2 = 10$ (since $9^{-1} = 2$)
- $Q_1(16) = 5 + 13 \cdot 10 = 135 \equiv 16$ ✓

Step 4 — FRI fold #2: down to a constant

- $\alpha_2 = \text{hash}(\text{root}_0 \,\|\, \text{root}_1) \bmod 17 = 6$.
- In domain $\{1, 13, 16, 4\}$, the pairs are $(1, 16)$ and $(13, 4)$, i.e. y and $-y$ again.
- Fold to the domain $\{1, 16\}$: values $[9, 9]$.

- Degree < 1 means **constant.** The final layer is tiny, so the prover sends it **in the clear.** The verifier checks directly that both entries are equal. ✓

Degree trail: $< 4 \to < 2 \to < 1$. Domain trail: $8 \to 4 \to 2$. Each fold halves both, and each one is the FFT butterfly run backwards (Part V.5).

Step 5 — The query phase: check the folds are honest

So far the verifier has three things: two roots and a final constant. The prover could have committed garbage at every layer and invented the constant. The queries tie the layers together.

- Query index from the transcript hash: $i = 2$.

Layer 0. Open positions 2 and 6 (the pair $x = 13, -x = 4$):

- Value 16 at index 2, with Merkle path `[aa4ca58e, cc36b119, dc7a368a]`
- Value 11 at index 6, with Merkle path `[3e44a2d3, e5cca044, dcb20c92]`
- Verifier rehashes each leaf up its path and gets root `4a4aa549` both times ✓ (*the values are the committed ones*)
- Verifier folds them itself with $\alpha_1 = 13 \to$ **expects 16** at layer-1 position 2.

Layer 1. Open positions 0 and 2 (the pair $y = 1, -y = 16$):

- Values 14 and 16, paths check out to `e35f743e` ✓
- Position 2 holds **16, matching the expected 16** ✓ (*fold #1 was done honestly at this point*)
- Fold with $\alpha_2 = 6$: $(14 + 16)/2 + 6 \cdot (14 - 16)/2 = 15 + 6 \cdot 16 = 111 \equiv 9 \to$ **expects 9** in the final layer.

Final layer. Is 9 ✓.

One query checks one thread through all layers. A real verifier repeats with ~30–80 independent query indices.

Step 6 — What a cheater runs into

Suppose the prover commits a corrupted codeword, changing $Q(8)$ from 12 to 13: `[2, 10, 16, 3, 0, 13, 11, 4]`. This is no longer a degree-< 4 evaluation.

Fiat-Shamir reacts first. The root changes (`552ebddb`), so every challenge changes: $\alpha_1 = 6$, $\alpha_2 = 10$. The prover can't pick the corruption after seeing the challenges.

Option A — fold honestly. Layer 1 becomes `[7, 2, 14, 5]`, and layer 2 becomes `[1, 3]`: *not constant*. The verifier rejects without making a single query. The degree problem rose to the surface.

Option B — lie in the layers to fake a constant. The cheater commits layer 1 as the fold of the *nearest true codeword*, `[7, 16, 14, 5]`, which folds to `[1, 1]`. Now everything looks fine except at one point: layer 0's pair (indices 1, 5) folds to **2**, but layer 1 claims **16**.

- A query catches it only if it lands on index 1 or 5: **2 out of 8 = 1/4 per query.**
- After s queries, the cheater survives with probability $(3/4)^s$. That's about 2×10^{-4} at 30 queries.

Two lessons from Option B:

1. This is **why queries exist**: folding forces the lie into the open *somewhere*, and random positions find it.
2. One corrupted value is the *close-to-a-codeword* case, and it's the hardest one to catch (1/4 here, not the $1 - \rho = 1/2$ from Part IV). A table far from every codeword gets caught much faster. Bounding exactly this behavior is what the **proximity-gap results** are about (Part IV caveat). It's also why real parameters use lower rates and more queries than this toy.

The whole protocol on one line

> **Commit** (evaluate on a $K\times$ domain, Merkle-root it) → **fold** $\log_2 d$ times with Fiat–Shamir α's, committing each layer → **send the final constant** → **open** s **random threads**, each checking Merkle paths and one fold equation per layer.

Proof size is $O(s \cdot \log^2 N)$ hashes: s queries × log N layers × a log N-length Merkle path. That's why STARK proofs are tens to hundreds of KB, compared with a few hundred bytes for KZG-based SNARKs.

What this example left out:

- **Evaluating Q at a point the verifier chooses** ("opening" in the polynomial-commitment sense). That's done by running FRI on $\frac{Q(X)-v}{X-z}$ (Fact 2 again), which is the **DEEP** technique listed under threads.
- **Batching:** many committed polynomials get combined by a random linear combination and folded together, not proven one at a time.
- **Folding factors larger than 2** (4, 8, 16) and stopping at a small polynomial instead of a constant, which shortens proofs.

Threads not yet covered

- **Fact 3:** division with remainder in $\mathbb{F}_p[X]$ → gcd, extended Euclid, exact vs. inexact division.
- **Fact 4:** evaluation ↔ interpolation as two views of one polynomial; **evaluations on a fixed domain = Reed–Solomon codeword.** The tutorial keeps the coefficient view; most other sources use the evaluation view — knowing they're the same object saves confusion. (Partly discharged in Part V.4.)
- **Fact 5:** degree arithmetic, and the bookkeeping of what bound each committed polynomial must meet.
- **FFT/NTT:** $f(X) = f_e(X^2) + X f_o(X^2)$ with $\omega^{i+n/2} = -\omega^i$. **FRI's folding is the same move:** take $f_e + \alpha f_o$ to halve the degree.

- **Multivariate polynomials:** AIR transition constraints in $2w$ variables; symbolic substitution of $t_i(X)$, $t_i(\omega X)$; a degree-k constraint on degree-$(n-1)$ trace polys gives degree $\approx k(n-1)$, which sets the blowup; Schwartz–Zippel in m variables.
- **Cosets** as a first-class topic: why quotient computation requires one, and why the vanishing polynomial is constant there.
- **DEEP / out-of-domain sampling:** why the verifier queries a point *outside* the evaluation domain before FRI begins.

Recall prompts

Answer from memory before re-reading. Each maps to a section above.

1. Which of the two STARK layers carries the soundness, and what replaces cryptography there? → I
2. Why does a prime modulus specifically buy you division? → II
3. Why is the trace placed on roots of unity rather than $\{0, 1, \ldots, n-1\}$? → II
4. State Fact 1 and derive the "two polynomials agree rarely" consequence. → III
5. A prover lies about one trace step. What *exactly* goes wrong with the quotient — and why is that the detectable thing? → III, V.3
6. Why can't you compute the quotient on the trace domain itself? → V.3
7. What bounds the evaluation domain size: p, or something else? → V.4
8. Rate $\rho = 1/8$, 30 queries. Soundness, and what caveat qualifies that number? → IV
9. Why must a protocol be public-coin to survive Fiat–Shamir? → VI
10. What is weak Fiat–Shamir, and why does it break soundness rather than completeness? → VII
11. A Merkle root commits to a vector. What does it *not* tell the verifier, and which step fills that gap? → VIII.2
12. From $f(x)$ and $f(-x)$ alone, how do you compute the folded value at x^2? Why does folding halve the degree? → VIII.3
13. A cheater corrupts one codeword value and fakes the next layer. Where does the inconsistency show up, and with what probability does one query catch it here? → VIII.6
14. Why does changing one committed value change every FRI challenge? → VIII.6